Module 11: Network Architecture Design

Module 11: Network Architecture Design

Overview

Every model before this one was a stack: one input tensor, layers in a line, one output. Real tabular data mixes numbers and categories, and this module treats the network as a graph wired around the data. Through a passenger-survival case study you move from the Sequential API to the Functional API, give each feature its own encoding path, merge them, and finish with a model that predicts two targets at once.

---

1. Baselines First

- **Majority class** — predict the most common label.
- **One-rule** — a single honest if-statement (here: sex).

A network that cannot beat one rule is decoration. On this dataset the one-rule baseline is embarrassingly strong, which is the point.

2. Sequential vs Functional

The Functional API calls layers on tensors; a model is whatever graph runs from named inputs to outputs. Stacks are the special case. `plot_model` draws the graph you wired — the drawing is the design document.

3. Front Doors for Every Column

Column type	Encoding	Width	Parameters
numeric, smooth effect	raw (+ Normalization)	1	0
numeric, non-smooth effect	Discretization buckets	#buckets	0

Column type	Encoding	Width	Parameters
small categorical	one-hot (Lookup layer)	vocab + OOV	0
large categorical	Embedding	dim	vocab × dim

4. Merging and the Bill

`Concatenate` joins the paths; the trunk costs `(width + 1) × units`. Read the trainable-parameter count like a bill — every wiring choice prices in, and the count should never surprise you. (`count_params()` also counts Normalization's frozen statistics; sum `trainable_weights` instead.)

5. Multiple Outputs

Several heads share one trunk; each has its own loss; training minimizes their weighted sum. Moving a column from input to target is a wiring decision with leakage consequences — a model must not be fed what it predicts.

The Tuning Thread

Regularization and hyperparameter tuning are not a lecture week in this course. They run through the assignments: learning curves, grid and random search, early stopping, keras-tuner. The module page keeps an interactive L2 dial in its Go deeper section as that thread's one-picture summary.

Files

- `lecture.ipynb` / `lecture.py` — the five-model ladder on the Titanic manifest
- `exercises/exercise_01.ipynb` — wire a six-input model; predict the parameter bill on paper
- `quiz.html`, `glossary.html`, `readings.html`, `discussion.html`